Design Decisions for the Completion Buffer

Block R #01: Completion Buffer, Retire Logic and MXIF Port

This document describes each design decision for the Completion Buffer (CB), organised by the case it handles. Signal names follow the R #01 draw.io block diagram.

---

1. Fence and Fence.i Case

1.1 `fence` handling

A `fence` requires all older load and store instructions to be architecturally committed, not merely retired. Loads pose no problem, but stores do.

Consider a `fence` at the head of the CB. The CB sends a drain request (`store_buffer_drain_o`) to the store buffer. The drain may be triggered by a full store buffer, or it may even be an incomplete drain. In either case, the data in the store buffer is drained to memory so that the older store operations are completed.

The `fence` at the head waits for the condition `store_buffer_empty_i && mxif_idle_i`. When this condition is satisfied, the `fence` retires.

`fence` and `fence.i` are valid instructions, so each of them increments the instruction count.

1.2 `fence.i` handling

Like `fence`, `fence.i` requires all older load and store instructions to be architecturally committed. It stays at the head of the CB until the condition `store_buffer_empty_i && mxif_idle_i` is satisfied. In addition, the following signals are used:

- `fencei_invalidate_I_cache_o`
- `fencei_invalidate_I_cache_done_i`
- `fencei_flush_pipeline_o`
- `fencei_redirect_valid_o`
- `fencei_redirect_pc_o` (this is simply the `next_pc` of the instruction, which is also reported as `rvfi_next_pc_o[63:0]`)
- `fencei_clean_D_cache_o`
- `fencei_clean_D_cache_done_i`

Three of these signals act as triggers to the core:

- `fencei_invalidate_I_cache_o` triggers the core to invalidate the I-cache.
- `fencei_flush_pipeline_o` triggers the core to flush the entire pipeline.
- `fencei_redirect_valid_o` triggers the core to route `next_pc` to the IF stage so that the next valid instruction is fetched.

The core also triggers the flushing of the instructions in the CB that are younger than the `fence.i` instruction, using `flush_cb_valid_i` and `flush_from_tag_i[2:0]`.

Cleaning the D-cache is required here; it is not optional. Otherwise the I-cache would fetch the old data from memory again, because the new copy is still sitting in the D-cache:

```
sw (new code)  ->  store buffer  ->  D-cache line (dirty)      memory: OLD code
fence.i        ->  I-cache invalidated -> refill from memory    -> OLD code
fetched
```

---

2. Branch Misprediction Case

Branch instruction handling

When a branch instruction reaches the EX stage and is resolved as mispredicted, the core itself flushes the instructions that were fetched on the wrong path (IF/ID) and provides the redirect PC to the IF stage, so that instructions are fetched from the correct PC.

Instructions younger than the branch have already been allocated in the CB, so these allocated instructions must be flushed from the CB as well. `flush_cb_valid_i` indicates the flush to the CB, and `flush_from_tag_i[2:0]` provides the tag from which to flush.

The branch instruction itself retires normally. In other words, the branch instruction is never flushed.

---

3. Trap Case

Fault / exception handling

When an exception is detected at the head of the CB, the following signals are captured:

- `trap_valid_o`
- `trap_cause_o[5:0]`
- `trap_tval_o[63:0]`
- `trap_pc_o[63:0]`
- `trap_interrupt_o` (0 = exception, 1 = interrupt)

The architectural register-file write-back, the CSR commit and the store-buffer commit are not performed.

The core takes the trap and jumps to the trap handler. All entries of the CB, including the trapped instruction at the head, are flushed internally. The CB also triggers the flush of the entire pipeline (`flush_pipeline_valid_o`). No instruction younger than the faulting one may change the architectural state of the core, so all younger instructions, together with the faulting one, are flushed from the pipeline as well.

The store buffer is also squashed (`store_buffer_squash_o`). The reason is that the entire CB is flushed. Consider a `sw` instruction that is younger than the trapped instruction: it has placed its data

in the store buffer, and that data would be sent to memory later. Because the instruction itself has been flushed, we do not want the data of an instruction that is no longer present in the CB to be stored. This is why the store buffer is squashed on a trap.

---

4. Forwarding Case

The CB is the fourth source of forwarding. A dependent instruction waits for the `done == 1` signal of the instruction it depends on.

For example, suppose a dependent `add` instruction depends on the result of a `mul` instruction. A `mul` takes almost 3 to 4 cycles to complete. Forwarding is performed only when the `mul` instruction has completed, and it does not matter whether the `mul` is at the head of the CB. The source instruction can be at any position in the CB, so as soon as the `mul` has completed and its `done == 1`, the forwarding is performed. That is, the result is transferred to the dependent `add`. The dependent instruction therefore always waits for the instruction it depends on to complete.

The same applies to an MXIF instruction. Suppose the `add` instruction depends on an MXIF instruction that has to write a scalar result back to a destination register of the core. The `add` waits until the coprocessor asserts `mxif_result_valid_i`. Note that what releases the dependent instruction is the arrival of the result, not the retirement of the MXIF instruction. Again, it does not matter whether the MXIF instruction is at the head of the CB.

The signals are defined as follows:

- `hit`: a matching entry exists (youngest match, `rd_we` set, `rd != x0`).
- `ready`: that entry's `done` bit is set.
- The ID stage stalls on `hit && !ready`.
- Priority order: EX/MEM first, then MEM/WB, then the CB.

The core sends both of its source registers to the CB: `rs1` on `fwd_rs1_addr_i[4:0]` and `rs2` on `fwd_rs2_addr_i[4:0]`. The CB compares these registers with the destination register of every instruction present in the CB. If a match is found, and the matched instruction's `done == 1`, then the following outputs are produced:

- for `rs1`: `fwd_rs1_hit_o`, `fwd_rs1_ready_o`, `fwd_rs1_data_o[63:0]`
- for `rs2`: `fwd_rs2_hit_o`, `fwd_rs2_ready_o`, `fwd_rs2_data_o[63:0]`

---

5. Multi-Cycle Case

Multiplication / division handling

Multi-cycle instructions such as MUL and DIV are dispatched from the main pipeline in the EX stage and handed over to the multiplier and divider modules. Both operations need more cycles to complete than the main-pipeline instructions.

For this reason the CB has separate ports for MUL and DIV. When an operation completes, its result is written, through its own port, to the CB entry at the tag that was allocated to that instruction.

When a `mul` instruction completes, it writes its result to the result field of its entry in the CB using:

- `mul_wr_valid_i`
- `mul_wr_tag_i[2:0]`
- `mul_wr_result_i[63:0]`

The same applies to `div`, using:

- `div_wr_valid_i`
- `div_wr_tag_i[2:0]`
- `div_wr_result_i[63:0]`

---

6. Main Pipeline Case

Main-pipeline instructions such as `add`, `sub`, branch, load, store, CSR update and store-buffer pointer update all update the CB through the write-back port.

- `wb_wr_valid_i` indicates that the write-back port has a valid update for the CB.
- `wb_wr_tag_i[2:0]` identifies the entry to be updated. The pipeline remembers the tag of the entry it allocated, so it can address that entry through the write-back port.
- `wb_wr_result_i[63:0]`: for a normal instruction that updates the register file, the result is written to the `result` field of the entry at that tag.
- `wb_wr_next_pc_i[63:0]` and `wb_wr_branch_taken_i` update the `next_pc` and `branch_taken` fields of the entry. In the WB stage the branch resolution and the correct `next_pc` are available.
- `wb_wr_csr_i` and `wb_wr_stp_i` update the CSR-update field and the store-buffer-pointer field of the same entry.
- If an exception occurs, for example a PMP violation or a memory misalignment, `wb_wr_exc_i`, `wb_wr_exccode_i[5:0]` and `wb_wr_exctval_i[63:0]` provide the exception indication, the exception code and the exception `tval`.

---

7. Entry Allocation Case

The allocation port is the only port that allocates an entry in the CB. The other ports only update entries that have already been allocated.

The CB entry contains the following fields:

`alloc_valid_i`, `alloc_done_i`, `alloc_rs1_data_i[63:0]`, `alloc_rs2_data_i[63:0]`, `alloc_rs_valid_i[1:0]`, `alloc_norollback_i`, `alloc_current_pc_i[63:0]`, `alloc_next_pc_i[63:0]`, `alloc_branch_taken_i`, `alloc_interrupt_i`, `alloc_instr_i[31:0]`, `alloc_rd_i[4:0]`, `alloc_rd_we_i`, `alloc_result_i[63:0]`, `alloc_exc_i`, `alloc_exccode_i[5:0]`, `alloc_exctval_i[63:0]`, `alloc_is_mxif_i`, `alloc_mxif_id_i[3:0]`, `alloc_csr_i` and `alloc_stp_i`.

All of the allocation information is packed into one struct, `alloc_cb_entry_t_i`. The CB then provides the tag assigned to that entry (`alloc_tag_o[2:0]`), and the pipeline remembers it so that it can update this entry later. The CB also provides a full indication (`cb_full_o`) so that the ID stage can stall the pipeline and no further entry is allocated in the CB.

The `cb_empty_o` signal

A `csrr` must read the value written by an older `csrw`. To guarantee this, the `csrr` is held in the ID stage until the `csrw` has retired, so that the `csrr` obtains the new value written by the `csrw`.

Without `cb_empty_o`, the `csrr` would read the old value. The `csrw` executes in the EX stage, but its result becomes visible only after the `csrw` retires from the CB, that is, after the commit. During this window (EX stage to CB retirement), if a `csrr` tries to read the CSR written by the `csrw`, it reads the old CSR value, because the new value is written only at retirement.

To protect against this, `cb_empty_o` informs the ID stage that the CB has retired the `csrw`. It is then safe for the `csrr` to read the CSR.

---

8. CSR Update and Store-Buffer Pointer Final Commit Case

`csr_update_t` is the CSR update held in a CB entry, and `store_buffer_ptr` is the store-buffer-related update held in a CB entry.

- `csr_update_t` comes from the EX stage and is carried to the WB stage, where it is assigned to the specified entry based on the tag.
- `store_buffer_ptr` is received by the pipeline in the MEM stage and is carried to the WB stage, where it is also assigned to the CB.

Cases in which the updates must not be committed

Trap at the head. Consider a trap on the instruction at the head of the CB. The `csr_update_t` and `store_buffer_ptr` updates held by entries that are younger than the trapped instruction must not be allowed to update their entries. On a trap, the register-file, `csr_update_t` and `store_buffer_ptr` updates are skipped, and all instructions, including the trapped one, are flushed from the CB. This is what a precise exception means.

Branch misprediction. Instructions on the wrong path may have computed `csr_update_t` and `store_buffer_ptr` updates. They are flushed in EX when the branch is resolved, and their updates must not reach the entries in the CB. In this case the CSR and store-buffer updates are skipped, and the CB entries that were on the wrong path are flushed. Unlike the trap case, the mispredicted branch entry, which is at the head, survives.

Trap on the head instruction itself. The `csr_update_t` or `store_buffer_ptr` may have been computed for the entry that is at the head, and that entry may then take a trap. In this case the normal commit of `csr_update_t` and `store_buffer_ptr` must never happen, because a trapped instruction is not retired normally: the trap path is taken instead.

Therefore `csr_update_t` and `store_buffer_ptr` are committed normally, except in these three scenarios:

1. **Trap at the head.** The updates belong to instructions younger than the trapped instruction, so they are never committed.
2. **Branch misprediction.** The updates belong to wrong-path instructions younger than the mispredicted branch, so they never happen and are not committed.
3. **Exception on the head instruction.** The head instruction itself has a `csr_update_t` or `store_buffer_ptr` update. This is also never committed, because on a trap the trap path is taken and the instruction is not committed. The retirement logic therefore checks for an exception before performing the normal CSR commit.

These three cases apply to `csr_update_t`, `store_buffer_ptr` and the register-file write.

Privilege mode

The CSR module provides `retire_csr_priv_mode_i[1:0]`. It is the privilege mode of the instruction currently at the head of the CB. It is safe to use this signal, because the CSR module holds the correct privilege-mode information.

---

9. Trap Handling Interface

On a trap, the CB provides the following information to the trap-handling module:

- `trap_valid_o`
- `trap_cause_o[5:0]`
- `trap_tval_o[63:0]`
- `trap_pc_o[63:0]`
- `trap_interrupt_o`

This is sufficient information for the trap handler to take the trap path.

---

10. Verification Interface

The verification interface captures information from the pipeline through the CB. It captures this information at the retirement of normal instructions, and also for traps, that is, for instructions that are never committed or retired normally. The captured information is sent to the debug module, where it is used to reconstruct the original PC for the verification of the core.

The interface consists of:

`rvfi_valid_o`, `rvfi_instr_o[31:0]`, `rvfi_pc_o[63:0]`, `rvfi_next_pc_o[63:0]`, `rvfi_branch_taken_o`, `rvfi_exception_o`, `rvfi_trap_cause_o[5:0]`, `rvfi_trap_tval_o[63:0]`, `rvfi_priv_mode_o[1:0]`, `rvfi_interrupt_o` and `rvfi_illegal_instr_o`.

---

11. Instruction Count Pulse

The CB triggers the valid-instruction-retirement signal `minstret_incr_o` to the performance counter module. A counter in that module counts the number of valid retired instructions.

---

12. MXIF Issue Channel

Valid and payload. The CB asserts `mxif_issue_valid_o` when the entry at the head is valid. `valid` must not depend combinationally on `ready`. The CB must keep `valid` asserted until the coprocessor asserts `mxif_issue_ready_i`. While `valid` is asserted, the CB keeps the following signals stable:

- `mxif_issue_instr_o[31:0]`
- `mxif_issue_mode_o[1:0]`
- `mxif_issue_pc_o[63:0]`
- `mxif_issue_id_o[3:0]`
- `mxif_issue_rs_o[2:0][63:0]`
- `mxif_issue_rs_valid_o[2:0]`

Transfer. The transfer happens in the cycle in which `mxif_issue_valid_o && mxif_issue_ready_i` is true.

Coprocessor response. In the same cycle, the coprocessor responds with the following signals:

- `mxif_issue_accept_i`: asserted combinationally. It means the coprocessor recognises the instruction. If the coprocessor never asserts it, the instruction is illegal.
- `mxif_issue_writeback_i`: indicates whether the coprocessor writes a scalar result back to the destination register of the entry held in the CB at the specified id/tag.
- `mxif_issue_dualwrite_i`: in version 1 it is hardwired to 0.
- `mxif_issue_loadstore_i`: tells the core that the coprocessor will access memory, and can read and write.

All four of these response signals must be provided in the same cycle as the issue handshake (`mxif_issue_valid_o && mxif_issue_ready_i`).

---

13. MXIF Commit Channel

In the cycle immediately after the condition `mxif_issue_valid_o && mxif_issue_ready_i` is satisfied, the core/CB drives:

- `mxif_commit_valid_o`
- `mxif_commit_id_o[3:0]`
- `mxif_commit_kill_o`

Note: a branch misprediction, or an older trap, squashes an MXIF entry that has not reached the head. No signal has ever been sent to the coprocessor for such an entry, so nothing has to be cancelled. This is why requirement R1.1 exists, and why `mxif_commit_kill_o` is tied to 0.

---

14. MXIF Retire Permission Channel

The coprocessor asserts `mxif_norollback_i` together with `mxif_norollback_id_i`. This tells the CB that the offloaded instruction will not cause a fault in the future. The CB can retire the instruction now, provided that it does not have to write anything back to the CB.

The other interface signals, such as RVFI and the instruction-count increment, are also driven at this point, because an MXIF instruction is also a valid instruction.

---

15. MXIF Result Channel

If the coprocessor has asserted `mxif_norollback_i`, the offloaded instruction will not trap in the future. The CB can then retire it if it does not have to write anything back to the CB.

If the instruction has to write a result back to the CB, the core waits for the following signals:

- `mxif_result_valid_i`
- `mxif_result_id_i[3:0]`
- `mxif_result_rd_i[4:0]`
- `mxif_result_data_i[63:0]`
- `mxif_result_we_i`

In case of an exception or trap, the coprocessor can also assert `mxif_result_exc_i` and `mxif_result_exccode_i[5:0]`. This is possible only before `mxif_norollback_i`, because after `norollback` the instruction is guaranteed never to cause an exception. Consequently, `mxif_norollback_i` always arrives together with `mxif_result_valid_i` or before it.

The CB informs the coprocessor that it accepts the result by asserting `mxif_result_cb_ready_o`. This means that the result provided by the coprocessor has been received correctly by the CB. The CB stores the result in its specified entry when the condition `mxif_result_valid_i && mxif_result_cb_ready_o` is true.

---

16. MXIF Extension (`mxif_idle_i`)

Consider an instruction that has been offloaded to the coprocessor. The coprocessor asserts `mxif_norollback_i`, and indicates through `mxif_issue_writeback_i` that no result will be written back to the CB. This means the instruction will not fault in the future and can be retired now. The CB therefore retires it.

Retirement does not mean that the execution of the instruction has completed. The coprocessor continues to execute it in the background, even though the CB has retired it and it no longer exists in the CB. The question is how the core/CB learns that the coprocessor has finished the background processing of a retired instruction. This is the purpose of `mxif_idle_i`: it informs the core that the coprocessor is now free.

It tells the CB: "I have no instruction in flight and no memory operation outstanding."

`mxif_idle_i` is therefore also the key signal for `fence` and `fence.i`, because it guarantees to the CB/core, from the coprocessor's side, that the coprocessor is completely drained. It is analogous to `store_buffer_empty_i` from the store buffer, which means that the store buffer has been completely drained to memory and all earlier operations are done.

Open Questions

1. Synchronisation between the two memory ports (LSU and coprocessor)

The memory has two ports, one for the LSU and one for the coprocessor, which creates a synchronisation problem. What happens when the coprocessor is accessing memory, for example while a vector store instruction is running, and a scalar `lw` instruction arrives? What does the scalar `lw` load if the vector store has not yet completed?

What the specification says:

> The rule (INTERFACES §1.5, repeated in the hazard table, spec §8.2). If a coprocessor memory operation is outstanding, a scalar load or store must stall. The hazard table says it is detected in the MEM stage with a variable penalty. It is described as a v1.0 platform-wide rule, simple and "obviously correct", costing "one comparator", and the text asks you to measure and report its cost.

Case under discussion. The `lw` reads the old memory contents even though the vector store has not yet been offloaded:

```
div     x1,...          # head of the CB, slow (30 cycles)
vse32.v v1,(a0)         # waits in the CB, not offloaded, loadstore unknown
lw      x5,0(a0)        # younger, reaches MEM at cycle 6
```

Cycle	What happens
6	`lw` reads address A in MEM, gets the old value, and parks it in its CB entry
31	`div` retires, `vse32.v` becomes the head and is offloaded, `loadstore = 1` arrives
50	The coprocessor writes the new data to A
55	`lw` retires with the old value in `x5`

2. D-cache cleaning

Does the hardware perform the D-cache cleaning, or does software perform it?